第14讲

SLAM：现在与未来

主要目标

1. 了解经典的 SLAM 实现方案。
2. 通过实验，比较各种 SLAM 方案的异同。
3. 探讨 SLAM 的未来发展方向。

前面介绍了一个 SLAM 系统中的各个模块的工作原理，这是研究者们多年工作的结晶。目前，除了这些理论框架，我们也积累了许多优秀的开源 SLAM 方案。不过，由于它们大部分的实现都比较复杂，不适合作为初学者的上手材料，所以我们放到了本书的最后加以介绍。相信读者通过阅读之前的内容，应该能明白其基本原理。

14.1 当前的开源方案

本讲是全书的总结，我们将带着读者去看看现有的 SLAM 方案能做到怎样的程度。特别地，我们重点关注那些提供开源实现的方案。在 SLAM 研究领域中，能见到开源方案是很不容易的。往往论文中介绍理论的内容只占 20%，余下的 80% 都写在代码中。正是这些研究者的无私奉献，推动了整个 SLAM 行业的快速前进，使后续研究者有了更高的起点。在我们开始做 SLAM 之前，应该对相似的方案有深入的了解，再进行自己的研究，这样才会更有意义。

本讲的前半部分将带领读者参观当前的视觉 SLAM 方案，评述其历史地位和优缺点。表 14-1 列举了一些常见的开源 SLAM 方案，读者可以选择感兴趣的方案进行研究和实验。限于篇幅，我们只选了一部分有代表性的方案。在后半部分，我们将探讨未来 SLAM 可能的发展方向，并给出一些当前的研究成果。

表 14-1 常见的开源 SLAM 方案

方案名称	传感器形式	地址
MonoSLAM	单目	https://github.com/hanmekim/SceneLib2
PTAM	单目	http://www.robots.ox.ac.uk/~gk/PTAM/
ORB-SLAM	单目为主	http://webdiis.unizar.es/~raulmur/orbslam/
LSD-SLAM	单目为主	http://vision.in.tum.de/research/vslam/lsdslam
SVO	单目	https://github.com/uzh-rpg/rpg_svo
DTAM	RGB-D	https://github.com/anuranbaka/OpenDTAM
DVO	RGB-D	https://github.com/tum-vision/dvo_slam
DSO	单目	https://github.com/JakobEngel/dso
VINS 系列	单目 + IMU 为主	https://github.com/HKUST-Aerial-Robotics/VINS-Mono
RTAB-MAP	双目/RGB-D	https://github.com/introlab/rtabmap
RGBD-SLAM-V2	RGB-D	https://github.com/felixendres/rgbdslam_v2
Elastic Fusion	RGB-D	https://github.com/mp3guy/ElasticFusion
Hector SLAM	激光	http://wiki.ros.org/hector_slam
GMapping	激光	http://wiki.ros.org/gmapping
OKVIS	多目 + IMU	https://github.com/ethz-asl/okvis
ROVIO	单目 + IMU	https://github.com/ethz-asl/rovio

14.1.1 MonoSLAM

说到视觉 SLAM，很多研究者第一个想到的是 A. J. Davison 的单目 SLAM 工作[2, 55]。Davison 教授是视觉 SLAM 研究领域的先驱，他在 2007 年提出的 MonoSLAM 是第一个实时的单目视觉

SLAM 系统[2]，被认为是许多工作的发源地①。MonoSLAM 以扩展卡尔曼滤波为后端，追踪前端非常稀疏的特征点。由于 EKF 在早期 SLAM 中占据着明显主导地位，所以 MonoSLAM 也是建立在 EKF 的基础之上，以相机的当前状态和所有路标点为状态量，更新其均值和协方差的。

图 14-1 所示是 MonoSLAM 在运行时的情形。可以看到，单目相机在一幅图像中追踪了非常稀疏的特征点（且用到了主动追踪技术）。在 EKF 中，每个特征点的位置服从高斯分布，所以我们能够以一个椭球的形式表达它的均值和不确定性。在该图的右半部分，我们可以找到一些在空间中分布着的小球。它们在某个方向上显得越长，越说明在该方向的位置不确定。我们可以想象，如果一个特征点收敛，我们应该能看到它从一个很长的椭球（相机在 Z 方向上非常不确定）变成一个小点的样子。

（a）　　（b）

图 14-1　MonoSLAM 的运行时截图。（a）追踪特征点在图像中的表示；（b）特征点在三维空间中的表示（见彩插）

这种做法在今天看来固然存在许多弊端，但在当时已经是里程碑式的工作了，因为在此之前的视觉 SLAM 系统基本不能在线运行，只能靠机器人携带相机采集数据，再离线地进行定位与建图。计算机性能的进步，以及用稀疏的方式处理图像，加在一起才使得一个 SLAM 系统能够在线地运行。从现代的角度看，MonoSLAM 存在诸如应用场景很窄，路标数量有限，稀疏特征点非常容易丢失的情况，对它的开发也已经停止，取而代之的是更先进的理论和编程工具。不过这并不妨碍我们对前人工作的理解和尊敬。

14.1.2　PTAM

2007 年，Klein 等人提出了 PTAM（Parallel Tracking and Mapping）[96]，这也是视觉 SLAM 发展过程中的重要事件。PTAM 的重要意义在于以下两点：

①这是他博士期间工作的延续。他现在也在致力于将 SLAM 小型化、低功率化。

1. PTAM 提出并实现了跟踪与建图过程的并行化。我们现在已然清楚，跟踪部分需要实时响应图像数据，而对地图的优化则没必要实时地计算。后端优化可以在后台慢慢进行，在必要时进行线程同步即可。这是视觉 SLAM 中首次区分出前后端的概念，引领了后来许多视觉 SLAM 系统的设计（我们现在看到的 SLAM 多半都分前后端）。
2. PTAM 是第一个使用非线性优化，而不是使用传统的滤波器作为后端的方案。它引入了关键帧机制：我们不必精细地处理每一幅图像，而是把几个关键图像串起来，然后优化其轨迹和地图。早期的 SLAM 大多数使用 EKF 滤波器或其变种，以及粒子滤波器等；在 PTAM 之后，视觉 SLAM 研究逐渐转向了以非线性优化为主导的后端。由于之前人们未认识到后端优化的稀疏性，所以觉得优化后端无法实时处理那样大规模的数据，而 PTAM 则是一个典型的反例。

   PTAM 同时是一个增强现实软件，演示了酷炫的 AR 效果（如图 14-2 所示）。根据 PTAM 估计的相机位姿，我们可以在一个虚拟的平面上放置虚拟物体，看起来就像在真实的场景中一样。

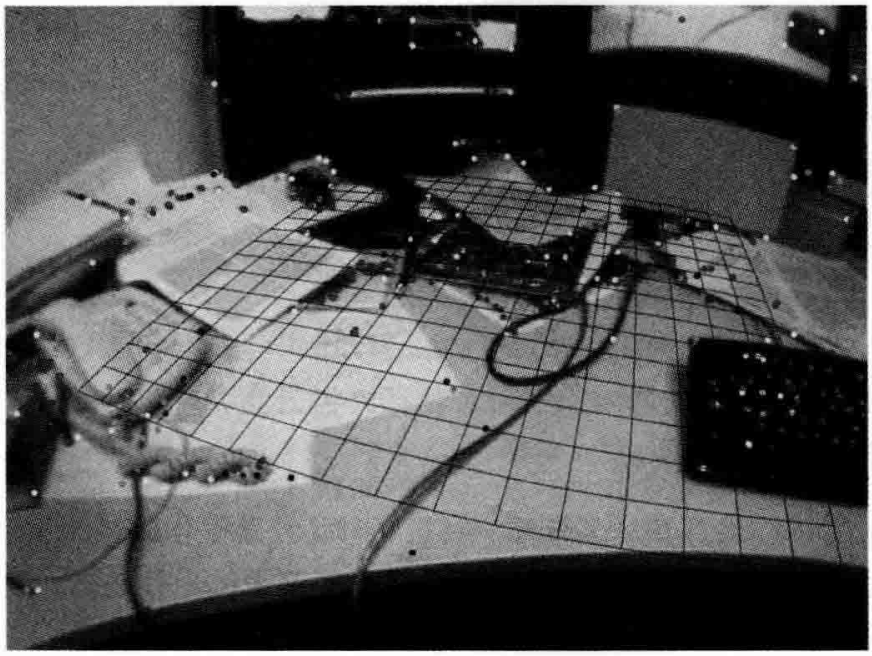


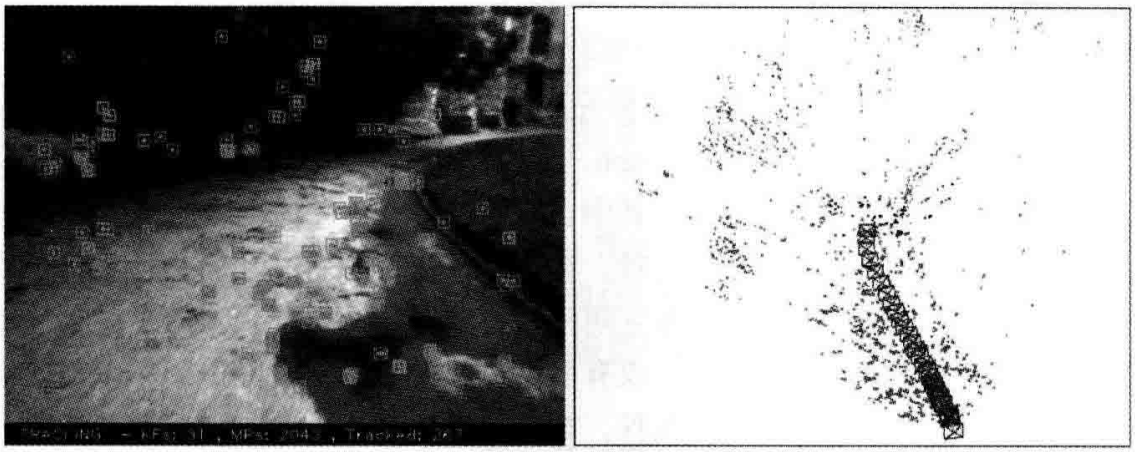
图 14-2 PTAM 的演示截图。它既可以提供实时的定位和建图，也可以在虚拟平面上叠加虚拟物体

PTAM 算是早期的结合 AR 的 SLAM 工作之一。与许多早期工作相似，存在着明显的缺陷：场景小，跟踪容易丢失，等等。这些又在后续的方案中得以修正。

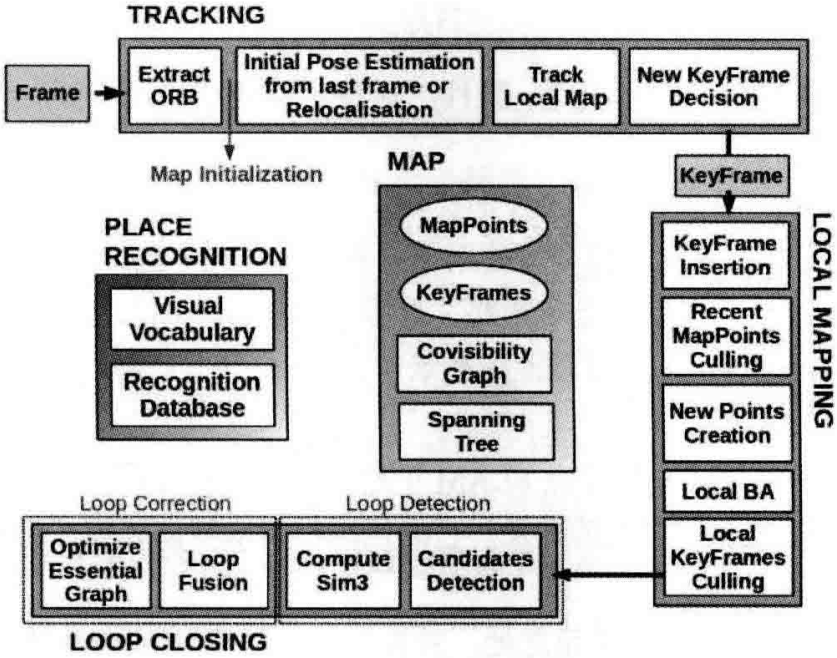
14.1.3 ORB-SLAM

介绍了历史上的几种方案之后，我们来看一些现代的 SLAM 系统。ORB-SLAM 是 PTAM 的继承者中非常有名的一位[88]（如图 14-3 所示）。它提出于 2015 年，是现代 SLAM 系统中做得非

常完善、非常易用的系统之一。ORB-SLAM 代表着主流的特征点 SLAM 的一个高峰。相比于之前的工作，ORB-SLAM 具有以下几条明显的优势。



（a）　　（b）



（c）

图 14-3　ORB-SLAM 运行截图。（a）为图像与追踪到的特征点，（b）为相机轨迹与建模的特征点地图，（c）为其标志性的三线程结构

1. 支持单目、双目、RGB-D 三种模式。这使得无论我们拿到了哪种常见的传感器，都可以先放到 ORB-SLAM 上测试，它具有良好的泛用性。
2. 整个系统围绕 ORB 特征进行计算，包括视觉里程计与回环检测的 ORB 字典。它体现出 ORB 特征是现阶段计算平台的一种优秀的效率与精度之间的折中方式。ORB 不像 SIFT 或 SURF 那样费时，在 CPU 上面即可实时计算；相比 Harris 角点等简单角点特征，又具有良好的旋转和缩放不变性。并且，ORB 提供描述子，使我们在大范围运动时能够进行

回环检测和重定位。

3. ORB 的回环检测是它的亮点。优秀的回环检测算法保证了 ORB-SLAM 有效地防止累积误差，并且在丢失之后还能迅速找回，许多现有的 SLAM 系统在这一点上做得不够完善。为此，在运行之前 ORB-SLAM 必须加载一个很大的 ORB 字典文件[①]。
4. ORB-SLAM 创新式地使用了三个线程完成 SLAM：实时跟踪特征点的 Tracking 线程，局部 BA 的优化线程（Co-visibility Graph，俗称**小图**），以及全局位姿图的回环检测与优化线程（Essential Graph 俗称**大图**）。其中，Tracking 线程负责对每幅新来的图像提取 ORB 特征点，并与最近的关键帧进行比较，计算特征点的位置并粗略估计相机位姿。小图线程求解一个 BA 问题，它包括局部空间内的特征点与相机位姿。这个线程负责求解更精细的相机位姿与特征点空间位置。仅有前两个线程，只能算完成了一个比较好的视觉里程计。第三个线程，也就是大图线程，对全局的地图与关键帧进行回环检测，消除累积误差。由于全局地图中的地图点太多，所以这个线程的优化不包括地图点，而只有相机位姿组成的位姿图。

   继 PTAM 的双线程结构之后，ORB-SLAM 的三线程结构取得了非常好的跟踪和建图效果，能够保证轨迹与地图的全局一致性。这种三线程结构也将被后续的研究者认同和采用。
5. ORB-SLAM 围绕特征点进行了不少的优化。例如，在 OpenCV 的特征提取基础上保证了特征点的均匀分布，在优化位姿时使用了一种循环优化 4 遍以得到更多正确匹配的方法，比 PTAM 更为宽松的关键帧选取策略，等等。这些细小的改进使得 ORB-SLAM 具有远超其他方案的稳健性：即使在较差的场景中，或标定的内参较差的情况下，ORB-SLAM 都能够顺利地工作。

上述优势使得 ORB-SLAM 在特征点 SLAM 中达到顶峰，许多研究工作都以 ORB-SLAM 作为标准，或者在它的基础上进行后续的开发。它的代码以清晰易读著称，有着完善的注释，可供后来的研究者进一步理解。

当然，ORB-SLAM 也存在一些不足之处。首先，由于整个 SLAM 系统都采用特征点进行计算，我们必须对每幅图像都计算一遍 ORB 特征，这是非常耗时的。ORB-SLAM 的三线程结构也给 CPU 带来了较重的负担，使得它只有在当前计算机架构的 CPU 上才能实时运算，移植到嵌入式设备上则有一定困难。其次，ORB-SLAM 的建图为稀疏特征点，目前还没有开放存储和读取地图后重新定位的功能（虽然从实现上讲并不困难）。根据我们在建图部分的分析，稀疏特征点地图只能满足我们对定位的需求，而无法提供导航、避障、交互等诸多功能。然而，如果我们仅用 ORB-SLAM 处理定位问题，似乎又显得过于重量级了。相比之下，另外一些方案提供了更为轻量级的定位，使我们能够在低端的处理器上运行 SLAM，或者让 CPU 有余力处理其他的事务。

①目前开源版 ORB-SLAM 使用了文本格式的字典，改成二进制格式字典之后可以加速不少。

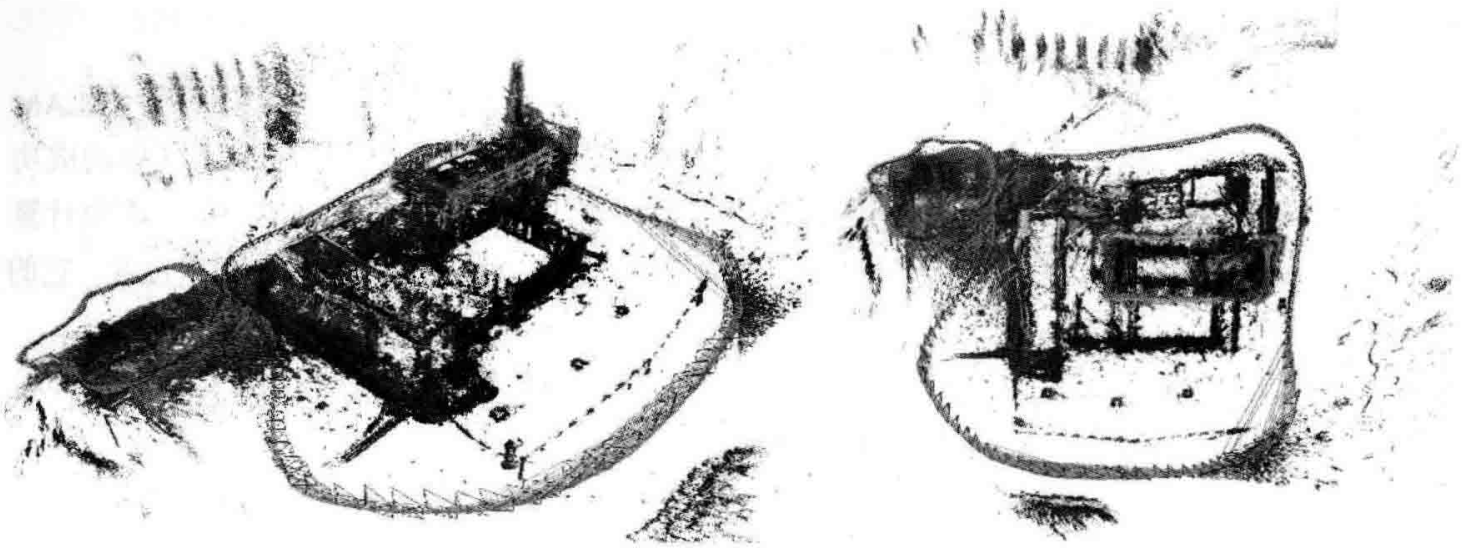
14.1.4　LSD-SLAM

LSD-SLAM（Large Scale Direct monocular SLAM）是 J. Engel 等人于 2014 年提出的 SLAM 工作[69, 75]。类比于 ORB-SLAM 之于特征点，LSD-SLAM 标志着单目直接法在 SLAM 中的成功应用。LSD-SLAM 的核心贡献是将直接法应用到了半稠密的单目 SLAM 中。它不仅不需要计算特征点，还能构建半稠密的地图——这里半稠密的意思主要是指估计梯度明显的像素位置。它的主要优点如下：

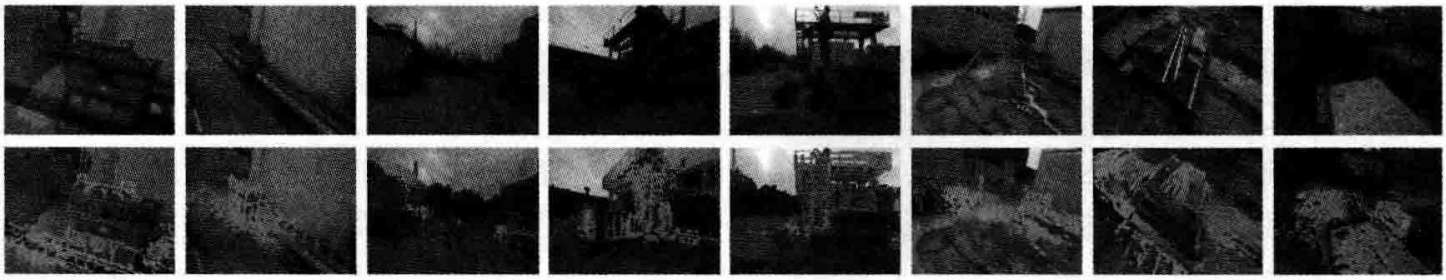
1. LSD-SLAM 的直接法是针对像素进行的。作者有创见地提出了像素梯度与直接法的关系，以及像素梯度与极线方向在稠密重建中的角度关系。这些在本书的第 8 讲和第 13 讲中均有讨论。不过，LSD-SLAM 是在单目图像进行半稠密的跟踪，实现原理要比本书的例程复杂。
2. LSD-SLAM 在 CPU 上实现了半稠密场景的重建，这在之前的方案中是很少见到的。基于特征点的方法只能是稀疏的，而进行稠密重建的方案大多要使用 RGB-D 传感器，或者使用 GPU 构建稠密地图[139]。TUM 计算机视觉组在多年对直接法研究的基础上，实现了这种 CPU 上的实时半稠密 SLAM。
3. 之前也说过，LSD-SLAM 的半稠密追踪使用了一些精妙的手段来保证追踪的实时性与稳定性。例如，LSD-SLAM 既不是利用单个像素，也不是利用图像块，而是在极线上等距离取 5 个点，度量其 SSD；在深度估计时，LSD-SLAM 首先用随机数初始化深度，在估计完后又把深度均值归一化，以调整尺度；在度量深度不确定性时，不仅考虑了三角化的几何关系，而且考虑了极线与深度的夹角，归纳成一个光度不确定性项；关键帧之间的约束使用了相似变换群及与之对应的李代数 $\zeta \in \mathfrak{sim}(3)$ 显式地表达出尺度，在后端优化中可以将不同尺度的场景考虑进来，减小了尺度飘移现象。

图 14-4 显示了 LSD-SLAM 的运行情况。我们可以观察这种微妙的半稠密地图是怎样一种介于稀疏地图与稠密地图之间的形式。半稠密地图建模了灰度图中有明显梯度的部分，显示在地图中，很大一部分都是物体的边缘或表面上带纹理的部分。LSD-SLAM 对它们进行跟踪并建立关键帧，最后优化得到这样的地图。看起来比稀疏的地图具有更多的信息，但又不像稠密地图那样拥有完整的表面（一般认为稠密地图无法仅用 CPU 实现实时性）。

由于 LSD-SLAM 使用了直接法进行跟踪，所以它既有直接法的优点（对特征缺失区域不敏感），又继承了直接法的缺点。例如，LSD-SLAM 对相机内参和曝光非常敏感，并且在相机快速运动时容易丢失。另外，在回环检测部分，由于目前并没有基于直接法实现的回环检测方式，因此 LSD-SLAM 必须依赖于特征点方法进行回环检测，尚未完全摆脱特征点的计算。



（a）



（b）

图 14-4 LSD-SLAM 的运行情况。（a）为估计的轨迹与地图；（b）为图像中被建模的部分，即具有较好的像素梯度的部分（见彩插）

14.1.5 SVO

SVO 是 Semi-direct Visual Odoemtry 的缩写[68]。它是由 Forster 等人于 2014 年提出的一种基于稀疏直接法的视觉里程计。按作者的称呼应该叫“半直接”法，然而按照本书的理念框架，称为“稀疏直接法”可能更好一些。**半直接**在原文中是指特征点与直接法的混合使用：SVO 跟踪了一些关键点（角点，没有描述子），然后像直接法那样，根据这些关键点周围的信息估计相机运动及其位置（如图 14-4 所示）。在实现中，SVO 使用了关键点周围的 4×4 的小块进行块匹配，估计相机自身的运动。

相比于其他方案，SVO 的最大优势是速度极快。由于使用稀疏的直接法，它既不必费力去计算描述子，也不必处理稠密和半稠密的信息，因此，即使在低端计算平台上也能达到实时性，在计算机上则可以达到每秒 100 多帧的速度。在后续的 SVO 2.0 中，速度更是达到了惊人的每秒 400 帧。这使得 SVO 非常适用于计算平台受限的场合，例如无人机、手持 AR/VR 设备的定位。无人机也是作者开发 SVO 的目标应用平台。

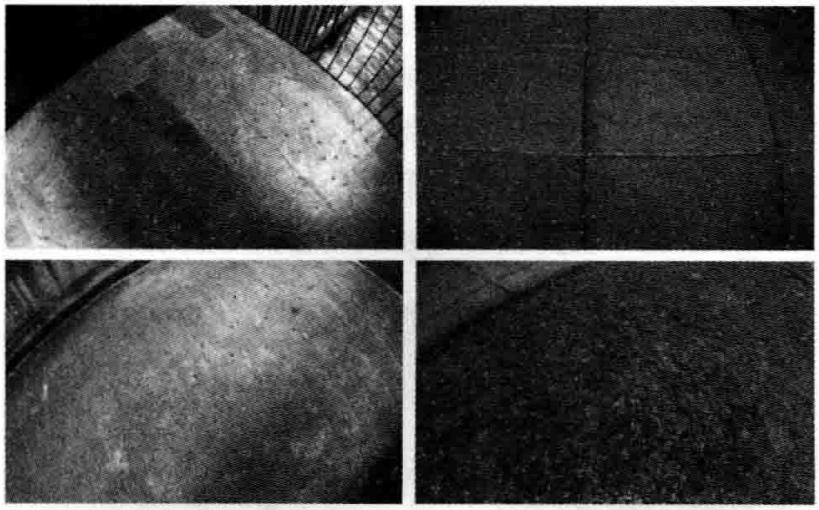


图 14-5　SVO 跟踪关键点的图片（见彩插）

SVO 的另一个创新之处是提出了深度滤波器的概念，并推导了基于均匀－高斯混合分布的深度滤波器。这在本书的第 13 讲有提及，但由于原理较为复杂，我们没有详细解释。SVO 将这种滤波器用于关键点的位置估计，并使用了逆深度作为参数化形式，使之能够更好地计算特征点位置。

开源版的 SVO 代码清晰易读，十分适合读者作为第一个 SLAM 实例进行分析。不过，开源版 SVO 也存在一些问题：

1. 由于目标应用平台为无人机的俯视相机，其视野内的物体主要是地面，而且相机的运动主要为水平和上下的移动，所以 SVO 的许多细节是围绕这个应用设计的，这使得它在平视相机中表现不佳。例如，SVO 在单目初始化时，使用了分解 $\boldsymbol{H}$ 矩阵而不是传统的 $\boldsymbol{F}$ 或 $\boldsymbol{E}$ 矩阵的方式，这需要假设特征点位于平面上。该假设对俯视相机是成立的，但对平视相机通常是不成立的，可能导致初始化失败。再如，SVO 在关键帧选择时，使用了平移量作为确定新的关键帧的策略，而没有考虑旋转量。这同样在无人机俯视配置下是有效的，但在平视相机中则容易丢失。所以，如果读者想要在平视相机中使用 SVO，则必须自己加以修改。
2. SVO 在追求速度和轻量化的同时，舍弃了后端优化和回环检测部分，也基本没有建图功能。这意味着 SVO 的位姿估计必然存在累积误差，而且丢失后不太容易进行重定位（因为没有描述子用来回环检测）。所以，我们称它为一个视觉里程计，而不是完整的 SLAM。

14.1.6　RTAB-MAP

介绍了几款单目 SLAM 方案后，我们再来看一些 RGB-D 传感器上的 SLAM 方案。相比于单目和双目，RGB-D SLAM 的原理要简单很多（尽管实现上不一定），而且能够在 CPU 上实时建立稠密的地图。

RTAB-MAP（Real Time Appearance-Based Mapping）[119] 是 RGB-D SLAM 中比较经典的一个方案。它实现了 RGB-D SLAM 中所有应该有的东西：基于特征的视觉里程计、基于词袋的回环检测、后端的位姿图优化，以及点云和三角网格地图。因此，RTAB-MAP 给出了一套完整的（但有些庞大的）RGB-D SLAM 方案。目前，我们已经可以直接从 ROS 中获得其二进制程序，此外，在 Google Project Tango 上也可以获取其 App（如图 14-6 所示）。

图 14-6　RTAB-MAP 在 Google Project Tango 上的运行样例（见彩插）

RTAB-MAP 支持一些常见的 RGB-D 和双目传感器，像 Kinect、Xtion 等，且提供实时的定位和建图功能。不过由于集成度较高，使得其他开发者在它的基础上进行二次开发变得困难，所以 RTAB-MAP 更适合作为 SLAM 应用而非研究使用。

14.1.7　其他

除了这些开源方案，读者还能在`openslam.org`之类的网站上找到许多其他的研究，例如，DVO-SLAM[140]、RGBD-SLAM-V2[97]、DSO[70]，以及一些 Kinect Fusion 相关的工作，等等。随着时代发展，更新颖、更优秀的开源 SLAM 作品也将出现在人们的视野中，限于篇幅这里就不逐一介绍了。

14.2　未来的 SLAM 话题

看过了现有的方案，我们再来讨论一些未来的发展方向①。大体上讲，SLAM 将来的发展趋势有两大类：一是朝轻量级、小型化方向发展，让 SLAM 能够在嵌入式或手机等小型设备上良好运行，然后考虑以它为底层功能的应用。毕竟，大部分场合中，我们的真正目的是实现机器人、

①这里有一部分是笔者个人的理解，不一定完全正确。

AR/VR 设备的功能，例如运动、导航、教学、娱乐，而 SLAM 是为上层应用提供自身的一个位姿估计。在这些应用中，我们不希望 SLAM 占用所有计算资源，所以对 SLAM 的小型化和轻量化有非常强烈的需求。二是利用高性能计算设备，实现精密的三维重建、场景理解等功能。在这些应用中，我们的目的是完美地重建场景，而对于计算资源和设备的便携性则没有多大限制。由于可以利用 GPU，这个方向和深度学习也有结合点。

14.2.1　视觉 + 惯性导航 SLAM

首先，我们要谈一个有很强应用背景的方向：视觉 – 惯性导航融合 SLAM 方案。实际的机器人也好，硬件设备也好，通常都不会只携带一种传感器，往往是多种传感器的融合。学术界的研究人员喜爱“大而且干净的问题”（Big Clean Problem），例如仅用单个摄像头实现视觉 SLAM。但产业界的朋友们则更注重让算法更加实用，不得不面对一些复杂而琐碎的场景。在这种应用背景下，将视觉与惯性导航融合进行 SLAM 成了一个关注热点。

IMU 能够测量传感器本体的角速度和加速度，被认为与相机传感器具有明显的互补性，而且极有可能在融合之后得到更完善的 SLAM 系统[141]。为什么这么说呢？

1. IMU 虽然可以测得角速度和加速度，但这些量都存在明显的漂移（Drift），使得积分两次得到的位姿数据非常不可靠。例如，我们将 IMU 放在桌上不动，用它的读数积分得到的位姿也会漂出十万八千里。但是，对于短时间内的快速运动，IMU 能够提供一些较好的估计。这正是相机的弱点。

   当运动过快时，（卷帘快门的）相机会出现运动模糊，或者两帧之间重叠区域太少以至于无法进行特征匹配，所以纯视觉 SLAM 非常害怕快速的运动。而有了 IMU，即使在相机数据无效的那段时间内，我们也能保持一个较好的位姿估计，这是纯视觉 SLAM 无法做到的。
2. 相比于 IMU，相机数据基本不会有漂移。如果相机放在原地固定不动，那么（在静态场景下）视觉 SLAM 的位姿估计也是固定不动的。所以，相机数据可以有效地估计并修正 IMU 读数中的漂移，使得在慢速运动后的位姿估计依然有效。
3. 当图像发生变化时，本质上我们没法知道是相机自身发生了运动，还是外界条件发生了变化，所以纯视觉 SLAM 难以处理动态的障碍物。而 IMU 能够感受到自己的运动信息，从某种程度上减轻动态物体的影响。

总而言之，我们看到 IMU 为快速运动提供了较好的解决方式，而相机又能在慢速运动下解决 IMU 的漂移问题——在这个意义下，它们二者是互补的。

当然，不管是理论还是实践，VIO（Visual Inertial Odometry，视觉惯性里程计）都是相当复杂的。其复杂性主要来源于 IMU 测量加速度和角速度这两个量的事实，所以不得不引入运动学计算。目前，VIO 的框架已经定型为两大类：松耦合（Loosely Coupled）和紧耦合（Tightly Coupled）[142]。松耦合是指 IMU 和相机分别进行自身的运动估计，然后对其位姿估计结果进行

融合。紧耦合是指把 IMU 的状态与相机的状态合并在一起，共同构建运动方程和观测方程，然后进行状态估计——这和我们之前介绍的理论非常相似。我们可以预见，紧耦合理论也必将分为基于滤波和基于优化两个方向。在滤波方面，传统的 EKF[143] 以及改进的 MSCKF（Multi-State Constraint KF）[144] 都取得了一定的成果，研究者对 EKF 也进行了深入的讨论（例如能观性[145]）；优化方面也有相应的方案[89, 146]。值得一提的是，尽管在纯视觉 SLAM 中优化方法已经占了主流，但在 VIO 中，IMU 的数据频率非常高，对状态进行优化需要的计算量就更大，因此目前仍处于滤波与优化并存的阶段[147, 148]。由于过于复杂，限于篇幅，这里就只能大概地介绍这个方向了。

VIO 为将来 SLAM 的小型化与低成本化提供了一个非常有效的方向（例如图 14-7 中提供了 VIO 相机硬件）。而且结合稀疏直接法，我们有望在低端硬件上取得良好的 SLAM 或视觉里程计效果，是非常有前景的。

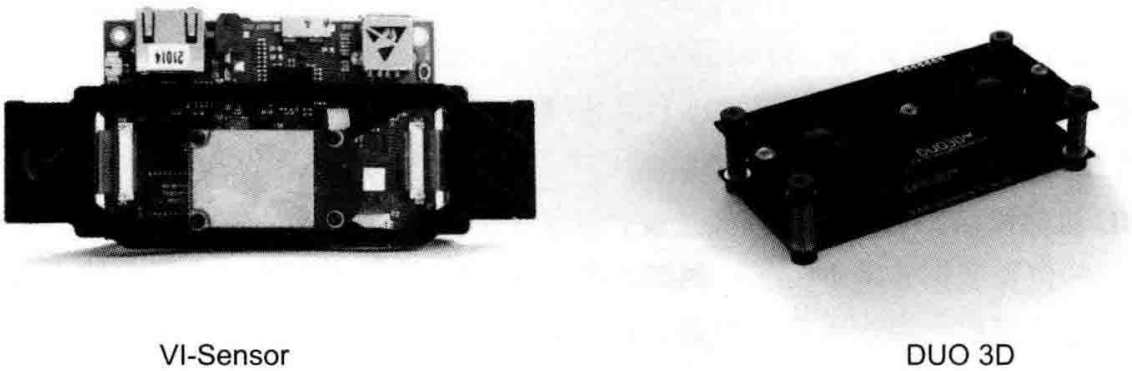


图 14-7　越来越多的相机开始集成 IMU 设备

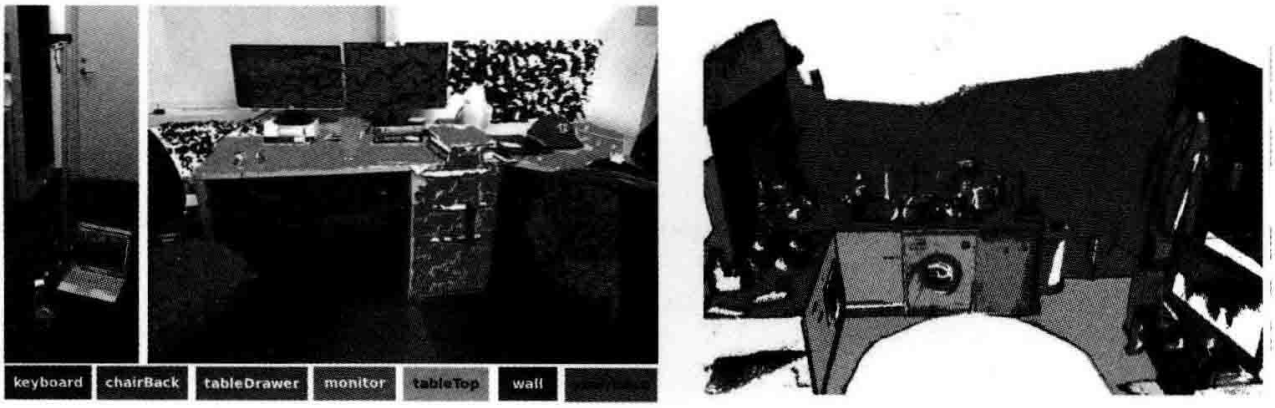
14.2.2　语义 SLAM

SLAM 的另一个大方向就是和深度学习技术结合。到目前为止，SLAM 的方案都处于特征点或者像素的层级，而关于这些特征点或像素到底来自于哪儿，我们一无所知。这使得计算机视觉中的 SLAM 与我们人类的做法不怎么相似，至少我们自己从来看不到特征点，也不会去根据特征点判断自身的运动方向。我们看到的是一个个物体，通过左右眼判断它们的远近，然后基于它们在图像中的运动推测相机的移动。

很久之前，研究者就试图将物体信息结合到 SLAM 中，如图 14-8 所示。例如，参考文献 [149–152] 中就曾把物体识别与视觉 SLAM 结合起来，构建带物体标签的地图。另外，把标签信息引入 BA 或优化端的目标函数和约束中，我们可以结合特征点的位置与标签信息进行优化[153]。这些工作都可以称为语义 SLAM。综合来说，SLAM 和语义的结合点主要有两个方面[9]：

1. 语义帮助 SLAM。传统的物体识别、分割算法往往只考虑一幅图，而在 SLAM 中我们拥有一台移动的相机。如果我们给运动过程中的图片都带上物体标签，就能得到一个带有标签的地图。另外，物体信息也可为回环检测、BA 优化带来更多的条件。

2. SLAM 帮助语义。物体识别和分割都需要大量的训练数据。要让分类器识别各个角度的物体，需要从不同视角采集该物体的数据，然后进行人工标定，非常辛苦。而 SLAM 中，由于我们可以估计相机的运动，可以自动地计算物体在图像中的位置，所以节省了人工标定的成本。如果有自动生成的带高质量标注的样本数据，则能够很大程度上加速分类器的训练过程。



（a）　　（b）

图 14-8　语义 SLAM 的一些结果，（a）和（b）分别来自文献 [152, 154]（见彩插）

在深度学习广泛应用之前，我们只能利用支持向量机、条件随机场等传统工具对物体或场景进行分割和识别，或者直接将观测数据与数据库中的样本进行比较[120, 154]，尝试构建语义地图[152, 155–157]。由于这些工具本身在分类正确率上存在限制，所以效果往往不尽如人意。随着深度学习的发展，我们开始使用网络，越来越准确地对图像进行识别、检测和分割[158–163]。这为构建准确的语义地图打下了更好的基础[164]。我们正看到，逐渐开始有学者将神经网络方法引入 SLAM 的物体识别和分割中，甚至 SLAM 本身的位姿估计与回环检测中[117, 165, 166]。虽然这些方法目前还没有成为主流，但将 SLAM 与深度学习结合来处理图像，也是一个很有前景的研究方向。

14.2.3　SLAM 的未来

除此之外，基于线/面特征的 SLAM[167–169]、动态场景下的 SLAM[170–172]、多机器人的 SLAM[82, 173, 174]，等等，都是研究者感兴趣并发力的地方。按照参考文献 [9] 的观点，视觉 SLAM 经过了三个大时代：提出问题、寻找算法、完善算法。而我们目前正处于第三个时代，面对着如何在已有的框架中进一步改善，使视觉 SLAM 系统能够在各种干扰的条件下稳定运行。这一步需要许多研究者的不懈努力。

当然，没有人能够预测未来，也没准突然有一天，整个框架都被新的技术推倒重写。不过即使是那样，今天我们的付出仍将是有意义的。没有今天的研究，就不会有将来的发展。最后，希望读者能在读完本书之后，对现有的整个 SLAM 系统有充分的认识。我们也期待你能够为 SLAM 研究做出贡献！

习题

1. 选择本讲提到的任意一个开源 SLAM 系统，在你的机器上编译运行它，直观体验其过程。
2. 你应该已经能够看懂绝大多数 SLAM 相关论文了。拿起纸和笔，开始你的研究吧！